An algebraic two-level Schwarz preconditioner

Design, performance studies and comparison with OpenFOAM GAMG
on the pressure equation of bubbly flows

Francesco Buffoli

Advanced Programming for Scientific Computing - Prof. L. Formaggia

Politecnico di Milano

October 2026

Abstract

This report describes a distributed-memory C++17/MPI implementation of the fully algebraic two-level overlapping Schwarz preconditioner of Al Daas, Jolivet and Rees [1]. The coarse space is built from local generalized eigenproblems that involve only the matrix, through a lumped local splitting of the subdomain matrices. The code is organized around an interface that mirrors the data an OpenFOAM processor owns (its rows and a global numbering), so that no process ever holds global data after the decomposition. Run-time selectable components provide exact and approximate (AMG) local solvers, sequential and distributed coarse solvers, GMRES and BiCGStab, and a reuse policy that updates the preconditioner along a sequence of matrices with the same pattern. The implementation reproduces the experiments of the reference paper on 240 processes. It was then applied to the pressure equation of a bubbly flow with density ratio 10^3, and compared with the GAMG solver of OpenFOAM on identical matrices of up to 32.8 million unknowns and 432 processes of the CINECA Galileo100 cluster. When every system is solved from zero to the tolerance used by OpenFOAM, the two-level method is 1.8 - 2.7× faster than GAMG and scales better; in a PIMPLE-like time loop with warm starts the advantage reduces to 0.96 - 1.81× per time step. The gain comes entirely from the number of iterations: the spectral coarse space captures the near-kernel modes associated with the bubbles, which the geometric agglomeration of GAMG cannot represent.

Contents

1 Introduction **2**

2 Problem and method **3**

2.1 The pressure equation of a bubbly flow 3

2.2 Overlapping Schwarz preconditioners 3

2.3 An algebraic spectral coarse space 4

2.4 Krylov methods and stopping tests 5

3 Software architecture **5**

3.1 Requirements and design principles 5

3.2 Code organization 5

3.3 Data distribution and communication 6

3.4 Class design of the preconditioner 7

3.5 The local eigensolver 9

3.6 Coarse assembly without global data 10
3.7 Setup reuse along a sequence of matrices 10
3.8 C++ techniques 11
3.9 Performance engineering 11
3.10 Build, tests and reproducibility 12

4 Results **13**
4.1 Platform and methodology 13
4.2 Reproduction of the reference paper 13
4.3 Optimizations for sequences of systems 14
4.4 Comparison with the GAMG solver of OpenFOAM 15
4.5 Interpretation 18

5 Discussion and future work **19**

6 Conclusions **19**

1 Introduction

In incompressible and low-Mach CFD solvers the pressure equation is solved several times per time step, and its solution is usually the most expensive part of the simulation. In two-phase flows the coefficient of this equation is the inverse of the density, which jumps by orders of magnitude across the interfaces: for air bubbles in water the ratio is about 10^3. The condition number of the matrix grows with that contrast, and the slowly converging error components are associated with the bubbles [2]. Multigrid methods whose coarse levels ignore the coefficients, as the geometric agglomeration used by default in the GAMG solver of OpenFOAM [3], cannot represent those components and leave them to the smoother.

Domain decomposition preconditioners with a spectral coarse space [4] are designed for exactly this situation: the coarse space is assembled from local eigenvectors that capture the low-energy modes of every subdomain, whatever their shape. The method of Al Daas, Jolivet and Rees [1] builds such a coarse space from the matrix alone, which makes it applicable inside a finite volume code that only assembles the global matrix. For symmetric positive definite and diagonally dominant matrices, the case of the pressure equation, the condition number of the preconditioned operator is bounded independently of the number of subdomains.

The project had four goals:

1. implement the preconditioner in C++ for distributed memory, and verify it against the results of the reference paper;
2. organize the code so that it can be fed by a CFD code that owns only its rows of the matrix (OpenFOAM), with no global data after the decomposition;
3. make it efficient for the use case of a CFD time loop, where hundreds of systems with the same sparsity pattern are solved;
4. measure, on identical matrices, whether it is competitive with the GAMG solver of OpenFOAM, the practical reference for pressure equations.

The work proceeded in phases. Each phase started from a measured bottleneck, was implemented with unit tests against sequential references, and was evaluated in a campaign on CINECA Galileo100 (G100) whose results, including the ideas that did not pay off, are recorded in the

repository[1].

Section 2 introduces the problem and the method. Section 3 describes the software architecture and the main implementation choices, Section 4 the verification and the numerical results, and Section 5 discusses the limitations and the next steps.

2 Problem and method

2.1 The pressure equation of a bubbly flow

The test problem used throughout the performance study is the pressure equation of an incompressible two-phase flow on the unit cube,

$$-\nabla \cdot (\beta \, \nabla p) = f, \qquad \beta = \frac{1}{\rho}, \qquad \rho = \alpha \, \rho_g + (1 - \alpha) \, \rho_l, \tag{1}$$

where α is the gas fraction of a field of rising bubbles with a smoothed interface about two cells thick, as produced by a volume-of-fluid method, and $\rho_l/\rho_g = 10^3$. The equation is discretized with cell-centred finite volumes on a uniform grid: the face coefficient is the arithmetic mean of β in the two neighbouring cells, walls are impermeable and $p = 0$ on the top face. The resulting matrix A is symmetric, positive definite, diagonally dominant and an M-matrix. The bubbles rise by half a cell per time step and wobble sideways, so a time loop produces a sequence of matrices with a fixed sparsity pattern whose values change near the interfaces. The same bubble field is evaluated by a small OpenFOAM application (Section 4.4), so that OpenFOAM assembles the same matrices.

The difficulty of (1) is easiest to see in terms of energy. A pressure field that is constant inside a bubble and varies only in the surrounding liquid has an energy $\int \beta |\nabla p|^2$ about 10^3 times smaller than a field of the same shape whose gradient lies inside the gas. Every bubble that does not touch the boundary where p is prescribed therefore carries one near-kernel mode. Figure 1 shows this on a two-dimensional example with five bubbles: the generalized eigenproblem $Au = \lambda \, \mathrm{diag}(A) \, u$ has exactly five eigenvalues (1.1×10^{-6} to 1.5×10^{-5}) that lie 33 to 430 times below the sixth one (4.8×10^{-4}), and their eigenvectors are nearly constant on each bubble. With a uniform density the gap disappears. A preconditioner that does not represent these modes on a coarse level can only reduce them through local smoothing, a few cells per iteration, while a bubble spans more and more cells as the mesh is refined; this is the behaviour reported for multigrid methods with coefficient-blind coarsening in [2].

2.2 Overlapping Schwarz preconditioners

The notation follows [1, 4]. The unknowns $\{1, \ldots, n\}$ are partitioned into N non-overlapping sets $\Omega_{I,i}$ by a graph partitioner applied to $\mathcal{G}(A + A^\top)$. Each set is extended by the nodes at distance one, $\Omega_{\Gamma,i}$, to obtain the overlapping subdomain $\Omega_i = [\Omega_{I,i}, \Omega_{\Gamma,i}]$ of size n_i. With the restriction $R_i = I_n(\Omega_i, :)$, the local matrix $A_{ii} = R_i A R_i^\top$ and a partition of unity, i.e. non-negative diagonal matrices D_i with $\sum_i R_i^\top D_i R_i = I_n$, the additive and restricted additive Schwarz preconditioners are

$$M_{\mathrm{ASM}}^{-1} = \sum_{i=1}^{N} R_i^\top A_{ii}^{-1} R_i, \qquad M_{\mathrm{RAS}}^{-1} = \sum_{i=1}^{N} R_i^\top D_i A_{ii}^{-1} R_i. \tag{2}$$

Both exchange information only between neighbouring subdomains, so the number of iterations grows with the number of subdomains, and a global correction is required [4, Ch. 4]. Given

[1] `https://github.com/fraBuffoli/Algebraic-Multilevel-Preconditioner`. Every campaign folder under `test/` contains the job scripts, the raw output of each job and a `RESULTS.md` summary.

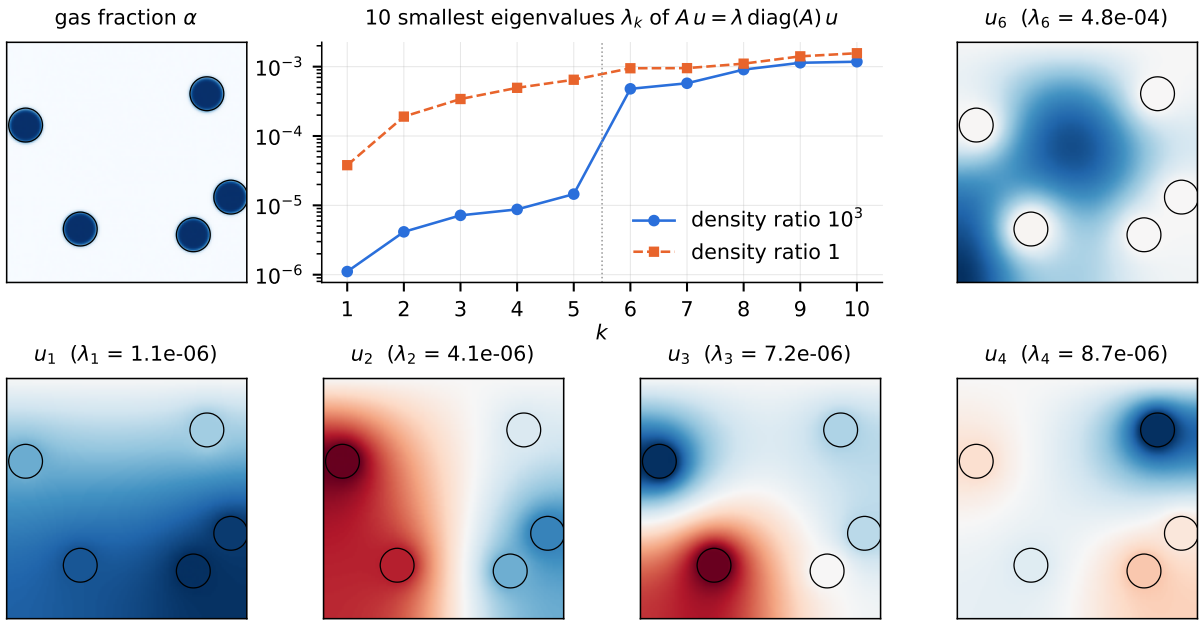


Figure 1: Near-kernel modes of the pressure matrix of a bubbly flow (2D, 128^2 cells, five bubbles, density ratio 10^3, $p = 0$ on the top side). Top: gas fraction, the ten smallest eigenvalues of $Au = \lambda\,\mathrm{diag}(A)\,u$ with density ratio 10^3 and 1, and the sixth eigenvector, the first one above the gap. Bottom: the first four eigenvectors, nearly constant inside every bubble (black contours: $\alpha = 0.5$). Matrices written by `matgen bubbles2d`, eigenpairs computed with SciPy (`fig/bubble_modes.py`).

$R_0 \in \mathbb{R}^{n_0 \times n}$ whose rows span a coarse space of dimension $n_0 \ll n$ and $A_{00} = R_0 A R_0^\top$, the two-level preconditioners with additive and deflated coarse correction are

$$M^{-1}_{\star,\mathrm{additive}} = R_0^\top A_{00}^{-1} R_0 + M_\star^{-1}, \tag{3}$$

$$M^{-1}_{\star,\mathrm{deflated}} = R_0^\top A_{00}^{-1} R_0 + M_\star^{-1}\left(I - A R_0^\top A_{00}^{-1} R_0\right), \tag{4}$$

with $\star \in \{\mathrm{ASM}, \mathrm{RAS}\}$. As in the experiments of [1], the default used in this work is RAS with the deflated correction.

2.3 An algebraic spectral coarse space

The coarse space of [1] is built from a local splitting of A obtained by lumping. With the local ordering $[\Omega_{I,i}, \Omega_{\Gamma,i}, \Omega_i^c]$, let s_i be the vector of the absolute row sums of the block $A(\Omega_{\Gamma,i}, \Omega_i^c)$, i.e. of the couplings of the overlap nodes with the outside of the subdomain. The local block splitting matrix is

$$\widetilde{A}_{ii} = \begin{pmatrix} A_{I,i} & A_{I\Gamma,i} \\ A_{\Gamma I,i} & A_{\Gamma,i} - \mathrm{diag}(s_i) \end{pmatrix}, \tag{5}$$

which differs from A_{ii} only on the diagonal of the overlap block and is therefore computed in $O(n_i)$ operations once the overlap rows are known. If A is symmetric positive definite and diagonally dominant, $\widetilde{A}_{ii}$ is a local positive semi-definite splitting of A [1, Lemma 3.2]. The local generalized eigenproblem is

$$\Pi_i D_i A_{ii} D_i \Pi_i\, u = \lambda\, \widetilde{A}_{ii}\, u, \tag{6}$$

where Π_i is the projection onto $\mathrm{range}(\widetilde{A}_{ii})$. With $L_i = \ker(D_i A_{ii} D_i)$ and $K_i = \ker(\widetilde{A}_{ii})$, the local contribution Z_i spans

$$(L_i \cap K_i)^{\perp_{K_i}} \;\oplus\; \mathrm{span}\,\{u \;:\; (6),\ |\lambda| \geq 1/\tau\}, \tag{7}$$

and the coarse space is spanned by the columns of $[R_1^\top D_1 Z_1, \ldots, R_N^\top D_N Z_N]$. The first term contains the eigenvectors of the infinite eigenvalues; it is not empty for floating subdomains, whose

lumped matrix has zero row sums and a non-trivial kernel. For symmetric positive definite and diagonally dominant matrices the condition number of the additive two-level operator satisfies

$$\kappa_2\Big(M^{-1}_{\mathrm{ASM,additive}}A\Big) \leq (k_c+1)\left(2+(2k_c+1)\frac{k_m}{\tau}\right), \tag{8}$$

where k_c is the number of colours needed to colour the subdomain graph and k_m the maximum number of subdomains sharing a row [1]: the bound does not depend on N nor on the coefficients, and the threshold τ trades the size of the coarse space for the condition number. In practice the number of vectors per subdomain is also capped by a parameter `nev`. For non-symmetric matrices the same construction is used without theoretical guarantee.

2.4 Krylov methods and stopping tests

The preconditioner is applied on the right inside restarted GMRES(30) [5], as in the reference paper, or BiCGStab [6], the method of the `PBiCGStab` solver of OpenFOAM. Right preconditioning keeps the stopping test on the unpreconditioned residual. Two tests are available: the 2-norm test $\|b-Ax\|_2 \leq \mathrm{rtol}\,\|b\|_2$ of the paper, and the normalised residual of the OpenFOAM solvers, $\sum|b-Ax|/F$ with $F=\sum|Ax_0-\bar{x}A\mathbf{1}|+\sum|b-\bar{x}A\mathbf{1}|$ and $\bar{x}$ the mean of the initial guess, with the `relTol` (relative) and `tolerance` (absolute) semantics of OpenFOAM. The second test is what makes the comparison with GAMG fair: both codes stop at the same accuracy in the same norm.

3 Software architecture

3.1 Requirements and design principles

The architecture follows from the target use, a preconditioner that a CFD code could call at every pressure solve:

- **Distributed by construction.** One MPI process owns one subdomain. After the decomposition no process holds global data: every global operation is a reduction, a neighbour exchange, or the assembly of the coarse matrix.
- **The interface of a CFD code.** The only input of the preconditioner is the set of rows owned by a process, with global column indices and contiguous ownership ranges. This is exactly what an OpenFOAM processor has: the `lduMatrix` of its cells, the coefficients of its processor patches and a `globalIndex`.
- **Algorithms and backends selected at run time.** Local and coarse direct solvers, approximate local solvers, Krylov methods and reuse policies are chosen from the command line, so that the same binary serves all campaigns; optional libraries are selected at compile time.
- **Testable against sequential references.** Every distributed component has a test that compares it with a reference computed from the global matrix on one process.
- **Measurable.** Every phase is timed by a scoped timer and reported as min/avg/max over the processes; every job records the sources and the environment it ran with.

3.2 Code organization

The library is split into six modules, each with its own directory under `include/` and `src/` and its own namespace, and each component with a header and a source file. Table 1 lists them and Figure 2 shows their dependencies. The structure is layered: a module only depends on the modules below it, and the drivers are the only code that sees all of them.

Table 1: Modules of the library (namespace `schwarz2lvl::<ns>`) and lines of code (headers + sources, comments included).

Module	ns	Main classes	Lines
`core`	`core`	scalar and index types, `SolverConfig` (command line), `LocalSystem`, MPI helpers	694
`partition`	`part`	`CsrMatrix`, `MatrixMarketIO`, `GraphPartitioner` (METIS), `DomainDecomposer`, `Subdomain`	1509
`linalg`	`linalg`	`HaloExchange`, `DistributedMatrix`, `LocalSolver` and its backends, AMG approximate inverse	1304
`preconditioner`	`prec`	`Preconditioner`, `OneLevelSchwarz`, `LocalEigensolver`, `CoarseSpace`, `CoarseSolver`, `TwoLevelSchwarz`, `ReusePolicy`	2702
`solver`	`solv`	`KrylovSolver`, `Gmres`, `BiCgStab`, `runTimeLoop`	927
`utils`	`utils`	`MatrixGenerator`, `BubbleField`, `TimerRegistry`, `Log`	843
drivers	-	`schwarz2lvl` (solver), `matgen` (test matrices)	262
tests	-	9 MPI test programs, about 500 checks	2680

3.3 Data distribution and communication

Decomposition. `DomainDecomposer` reproduces the `decomposePar` step of OpenFOAM: rank 0 reads a Matrix Market file or runs a built-in generator, partitions $\mathcal{G}(A + A^\top)$ with METIS [7], renumbers the unknowns so that process p owns the contiguous range $[o_p, o_{p+1})$, and sends to each process only its rows. For systems with several unknowns per cell the graph is compressed by cells, so that the unknowns of a cell are never split. The result is a `LocalSystem`, the only interface between the decomposition and the preconditioner:

```
struct LocalSystem {
    MPI_Comm comm;                       // communicator of the decomposition
    GlobalIndex n_global;                // global number of unknowns
    std::vector<GlobalIndex> ownership;  // rank p owns [ownership[p], ownership[p+1])
    std::vector<GlobalIndex> row_ptr, col; // owned rows in CSR, global column indices
    std::vector<Scalar> val, rhs;        // values and owned right-hand side
    int block_size; bool symmetric;
};
```

Global indices are 64-bit and local ones 32-bit, so that large problems do not inflate the memory of the local matrices.

Overlap and halo. `Subdomain` builds everything a process knows about its overlapping subdomain, collectively but without global data. The overlap $\Omega_{\Gamma,i}$ consists of the off-process columns of the owned rows (A part) and of the remote rows that have a column in $\Omega_{I,i}$ ($A^\top$ part); the latter are discovered with one `MPI_Alltoall` of counts and one `MPI_Alltoallv`, which makes the neighbour relation symmetric. The owners then send the full rows of the ghost unknowns, the only extra matrix data the method needs, and the local matrices A_{ii} (column major, to be factorized) and $A(\Omega_{I,i}, \Omega_i)$ (row major, for the products) are assembled; the lumping sums s_i of (5) are accumulated while the ghost rows are received. The local numbering places the owned unknowns first, in global order, followed by the ghosts sorted by global index (Figure 3). Since ownership ranges are contiguous, the ghosts owned by one neighbour form a contiguous block, which messages fill directly without unpacking.

Communication patterns. `HaloExchange` implements the two operators of the method with non-blocking point-to-point messages to the neighbours only: `forward()` realizes R_i (every ghost receives the owner's value) and `reverseAdd()` realizes $\sum_j R_j^\top$ (ghost values are sent back and

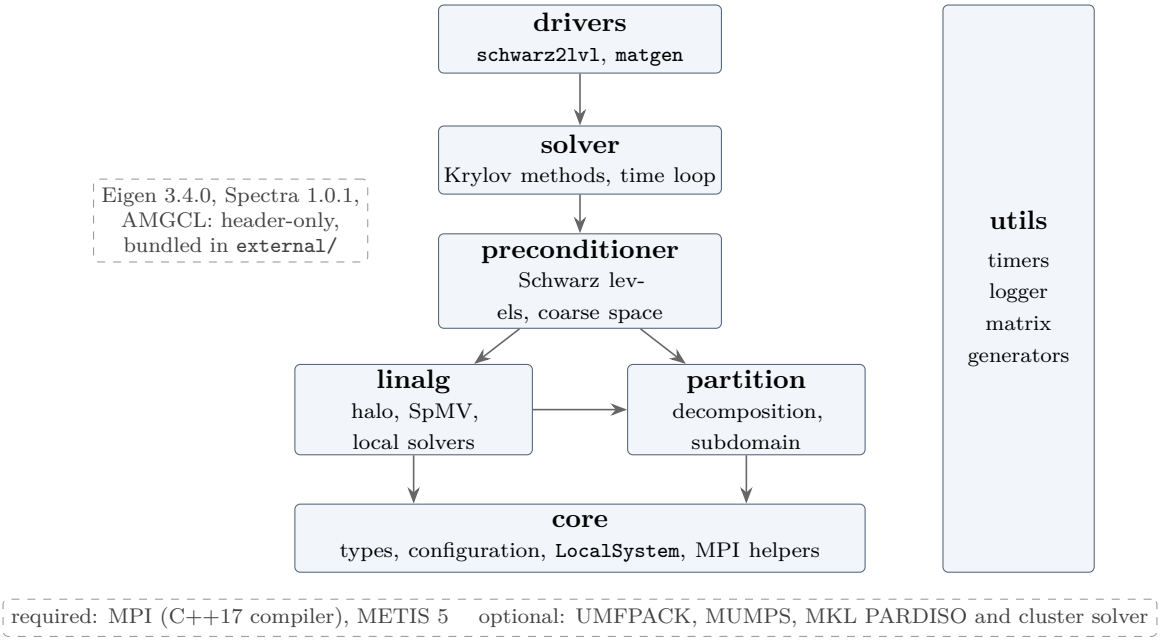


Figure 2: Module dependencies (an arrow reads "uses"). `linalg` depends on `partition` only through the `Subdomain` wrapped by `DistributedMatrix`. `utils` is cross-cutting: every module uses its timers and logger, and the decomposition calls its matrix generators.

added to the owned ones). A templated `exchange<T>()` sends irregular data (ghost rows, sparse rows of the coarse basis) with a size handshake. `DistributedMatrix` computes $y = Ax$ as one `forward()` followed by a local product with the owned rows, the scheme of `Amul` in OpenFOAM. Distributed vectors store only the owned entries, so inner products are local products followed by one `MPI_Allreduce`. With the Boolean partition of unity ($D_i = 1$ on $\Omega_{I,i}$, 0 on the ghosts), RAS needs no reverse communication and the restriction and prolongation of the coarse correction are purely local: one application of (4) costs two `forward()` exchanges (local solve and extra product) and the collectives of the coarse solve.

3.4 Class design of the preconditioner

Figure 4 shows the main classes. The design combines dynamic polymorphism, where a choice is made at run time and the cost of a virtual call is negligible against the work it triggers (a sparse solve or a coarse correction), with templates, where variants differ only in types known at compile time (precision, Eigen solver, MPI datatype).

Preconditioner. The abstract class `Preconditioner` exposes the application $z = M^{-1}r$ on owned vectors and an `update(UpdateLevel)` method that brings the preconditioner up to date with new matrix values (Section 3.7). `OneLevelSchwarz` implements (2) with one `LocalSolver` for A_{ii}. `TwoLevelSchwarz` is composed of a `OneLevelSchwarz`, the result of the `LocalEigensolver` and a `CoarseSpace`, which owns the `CoarseSolver` of A_{00}; its `apply()` is a direct transcription of (3) - (4) (deflated variant, timers omitted):

```
coarse_->apply(r, x0_);          // x0 = R0^T A00^{-1} R0 r
A_.apply(x0_, w_);               // deflated variant: one extra product
w_ = r - w_;
one_->apply(w_, z1_);            // M_RAS^{-1} (r - A x0)
z = x0_ + z1_;
```

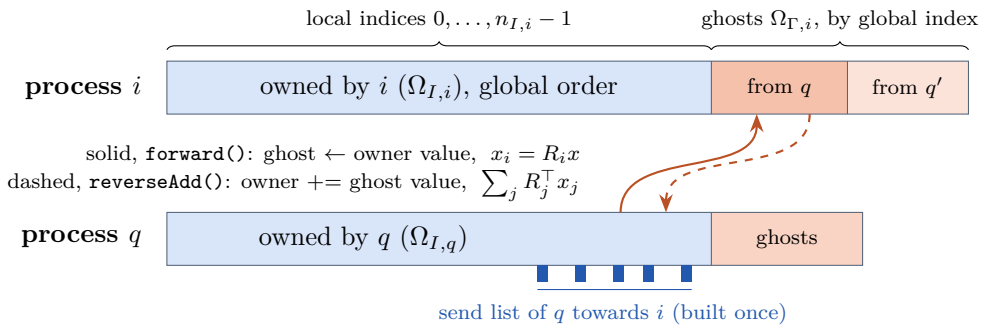


Figure 3: Local numbering of the vectors and the two operations of `HaloExchange`. On process i the owned unknowns come first and the ghosts follow, sorted by global index: with contiguous ownership ranges the ghosts owned by one neighbour form a contiguous block, which a message fills in place. The entries a process sends to each neighbour (its send lists) are computed once, when the subdomain is built; both operations use non-blocking point-to-point messages between neighbours only.

Local solvers. `LocalSolver` is a strategy interface for sequential sparse solvers: `factorize()`, `refactorize()` (numerical factorization reusing the symbolic analysis, with a fallback to a full factorization), multi right-hand-side `solve()` and `solveTranspose()`. The backends are the Eigen solvers [8] (SparseLU, SimplicialLLT, SimplicialLDLT), UMFPACK [9] called directly on the compressed arrays of the Eigen matrix, MKL PARDISO [10], and one V-cycle of AMGCL [11] as an approximate inverse. Precision is a template parameter: with `float` the matrix is converted once, the right-hand side is rounded and the solution converted back, and `if constexpr` removes every conversion in the `double` instantiation. `ChainSolver` composes solvers of decreasing requirements (Cholesky, then $\mathrm{LDL}^\top$, then LU): it tries each candidate until one factorizes, and later refactorizations start from the one that worked. A symmetric factorization is requested only if the matrix is known to be symmetric, since it reads only one triangle. The factory `makeLocalSolver(spec)` builds the right object from a small specification (backend, request, symmetry, precision, approximation).

Coarse solvers. `CoarseSolver` receives the coarse matrix as distributed triplets whose duplicates must be summed. `RootCoarseSolver` gathers them on rank 0 and factorizes with a `LocalSolver` chain; `MumpsCoarseSolver` passes them to MUMPS [12] as a distributed assembled matrix, optionally on a subset of leader processes; `CPardisoCoarseSolver` uses the cluster version of PARDISO with distributed rows, and checks every distributed Cholesky factorization with a probe solve, because with several processes MKL does not report a matrix that is not positive definite. The choice between a distributed solver and rank 0 is made collectively from n_0: for small coarse problems the fixed cost of the collectives of a distributed solve dominates.

Krylov solvers. `KrylovSolver` has two implementations. `Gmres` orthogonalizes with classical Gram - Schmidt applied twice (CGS2), so that the j inner products of a pass need a single reduction (three reductions per iteration instead of $j + 1$ with modified Gram–Schmidt), and accepts convergence only if the true residual, recomputed explicitly, is within 10% of the tolerance. With the OpenFOAM test it reconstructs $\sum |r_j|$ from the Krylov basis and the Givens rotations at every iteration. `BiCgStab` follows the algorithm of OpenFOAM's `PBiCGStab` (right preconditioning, stopping test at the half step) with the reductions fused into four per step instead of six. In both, an iteration is an application of the preconditioner, so that iteration counts and costs compare directly.

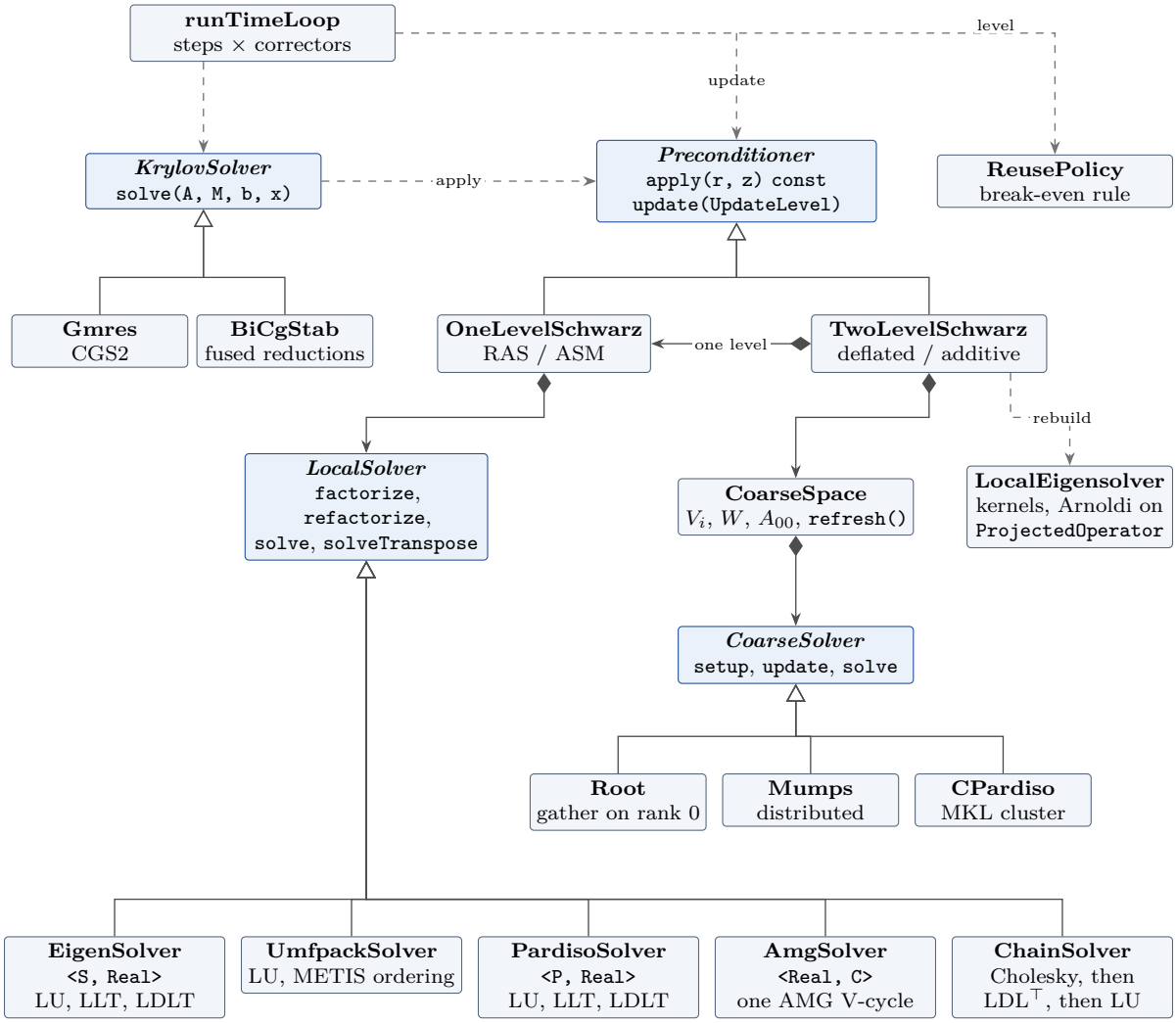


Figure 4: Main classes of the preconditioner and the solvers (italic: abstract interfaces). Open triangles denote inheritance, diamonds ownership through `std::unique_ptr`, dashed arrows use. Concrete objects are created by factories (`makeLocalSolver`, `makeCoarseSolver`, `makeKrylovSolver`) from the configuration; the template parameters select the Eigen or PARDISO solver type, the precision of the factors (`double` or `float`) and the AMG coarsening. `LocalEigensolver` and `Root` factorize their matrices through a `ChainSolver`. The data classes of Section 3.3 (`Subdomain`, `HaloExchange`, `DistributedMatrix`), which all of them use, are omitted.

3.5 The local eigensolver

$\widetilde{A}_{ii}$ is singular for every floating subdomain, so it can never be factorized as it is, and the infinite eigenvalues of (6) must be separated from the finite ones. `LocalEigensolver` does this explicitly:

1. factorize $\widetilde{A}_{ii} + \sigma I$ with $\sigma = 10^{-13}\|\widetilde{A}_{ii}\|$, a regularization at the level of round-off (Cholesky chain for symmetric matrices);
2. detect the kernel K_i by block inverse iteration with that factorization (three multi right-hand-side solves, block enlarged while all probes lie in the kernel), followed by an SVD of $\widetilde{A}_{ii}Y$; for non-symmetric matrices the left kernel is computed in the same way with transposed solves;
3. compute $(L_i \cap K_i)^{\perp_{K_i}}$ exactly from the SVD of B_iQ, $B_i = D_iA_{ii}D_i$, instead of relying on eigenvalues "close to ε^{-1}";
4. compute the largest finite eigenvalues with the implicitly restarted Arnoldi method of Spectra [13], applied matrix-free to $T = \Pi_K(\widetilde{A}_{ii} + \sigma I)^{-1}\Pi_R B_i \Pi_K$, where Π_K and Π_R project

Table 2: What each part of the setup depends on and what each update level recomputes. The basis V_i is kept by a refactorization: it was computed for the matrix of the last rebuild.

Depends on	Data	Code	Reuse	Refactorize	Rebuild
graph of A	overlap, halo, patterns, D_i	`Subdomain` (constructor)	kept	kept	kept
values of A	A_{ii}, $A(\Omega_I, \Omega_i)$, s_i	`Subdomain::updateValues`	kept	refilled	refilled
pattern of A_{ii}	symbolic factorization of A_{ii}	`LocalSolver`	kept	kept	redone
values of A	numerical factorization of A_{ii}	`LocalSolver`	kept	redone	redone
values of A	$\widetilde{A}_{ii}$, eigenproblems, V_i, W	`LocalEigensolver`	kept	**kept (old)**	redone
A and V	A_{00} and its factorization	`CoarseSpace::refresh`	kept	redone	redone

out the right and left kernels: each application costs one SpMV and one pair of triangular solves.

The operator T is a small adapter class, `ProjectedOperator`, with the interface that Spectra expects (`rows()`, `cols()`, `perform_op()`); it also counts and times its applications. Only eigenvalues with $|\lambda| \geq 1/\tau$ are kept, so instead of computing `nev` eigenpairs at once the solver starts with $k = 32$ and enlarges k until the smallest computed modulus falls below $1/\tau$: the selected vectors are the same, but the cost follows the number of selected vectors. A complex pair contributes the real and imaginary parts of its eigenvector, which span the same real invariant subspace, so the whole code works in real arithmetic. Subdomains with fewer than 400 unknowns use a dense path (SVD and full eigendecomposition), which is also the reference of the tests.

3.6 Coarse assembly without global data

`CoarseSpace` orthonormalizes $D_i Z_i$ with a column-pivoted QR, which keeps the span, drops dependent vectors and gives a better conditioned A_{00}. Process i owns the m_i coarse unknowns generated by its basis V_i; their global numbering needs one `MPI_Allgather`. The coarse matrix $A_{00} = R_0 A R_0^\top$ with $R_0^\top = [R_1^\top V_1, \ldots, R_N^\top V_N]$ is assembled from local products only:

$$A_{00} = \sum_{i=1}^{N} W(\Omega_{I,i}, :)^\top \, A(\Omega_{I,i}, \Omega_i) \, W(\Omega_i, :), \tag{9}$$

where $W = R_0^\top$ restricted to Ω_i is obtained with one forward exchange of sparse rows. Each process contributes the triplets of its term, and the coarse solver sums them. Since W depends only on the basis, it is kept: when the values of A change but the basis does not, `refresh()` recomputes (9) with two local products and the coarse solver only refactorizes numerically.

3.7 Setup reuse along a sequence of matrices

In a CFD time loop the sparsity pattern of the pressure matrix never changes, its values change slightly at every step and not at all between the correctors of one step. Creating a new preconditioner at every solve throws away work that depends only on the pattern. The setup is therefore split by what each part depends on (Table 2), and `update()` redoes only what is needed at one of three levels.

`Subdomain::updateValues()` computes once, at construction, the position in the value arrays of the local matrices of every owned and received coefficient; afterwards a change of values costs one halo exchange of the ghost-row values (no indices) and an indexed copy. A `Refactorize` keeps the coarse basis of the last rebuild but recomputes A_{00} with the current matrix, so the preconditioner remains a valid preconditioner of the new matrix; only its coarse space is tuned to an older one, and the iterations grow as the bubbles move away.

When to rebuild is decided by `ReusePolicy` with a break-even rule. Let q_{ref} be the number of iterations per decade of residual reduction of the first solve after the last rebuild. Every later solve with k iterations and d decades wastes $(k - q_{\mathrm{ref}}\, d)\, t_{\mathrm{it}}$ seconds because of the old basis; a rebuild is requested when the time wasted since the last rebuild reaches the extra cost of a rebuild over a refactorization, both measured during the run. This is the classical rent-or-buy decision: it adapts by itself to the machine, the subdomain size and the contrast, with no threshold to tune. Two details made it robust on the cluster: the costs are the lower medians of all the measured rebuilds and refactorizations, so that a single stalled factorization does not freeze the rebuilds, and with warm starts or different tolerances per corrector each corrector index keeps its own q_{ref}. Since all decisions are local floating-point computations, `update()` first agrees on the maximum requested level over all processes: a process refactorizing while another rebuilds would deadlock in the collectives of the coarse solver.

3.8 C++ techniques

- **Ownership and RAII.** Components are owned through `std::unique_ptr` and created by factories; resources of external libraries are released in destructors (UMFPACK symbolic and numeric objects, MUMPS instances, MPI communicators created with `MPI_Comm_split`). `ScopedTimer` measures the lifetime of a scope and accumulates it in a registry, so timing a phase is a one-line declaration that cannot be left unbalanced by an early return or an exception.
- **Templates where types vary.** The trait `mpiType<T>()` maps C++ types to MPI datatypes, so that generic helpers (`allreduceSum<T>`, `HaloExchange::exchange<T>`) are type safe; the solver backends are class templates over the Eigen or PARDISO solver type and over the precision.
- **Collective error handling.** An exception thrown on one process while the others enter a collective would hang the job. Failures that can occur on a subset of processes (a factorization, a local eigensolve, the decomposition on rank 0) are therefore turned into a status that is reduced over all processes before any of them throws, so every process reports the error.
- **Configuration.** `SolverConfig` parses the command line through a table that maps each option name to a lambda setting the corresponding field, followed by a `validate()` step: adding an option is one line, and invalid combinations are rejected before any MPI work starts.
- **Documentation.** Every class and function is documented with Doxygen comments that refer to the equations of the paper (`make doc`).

3.9 Performance engineering

Several implementation details changed the cost by large factors without changing the preconditioner; each was identified from the timers and verified to leave n_0 and the iteration counts unchanged:

- **No iterative refinement in the direct solvers.** The Eigen interface of PARDISO enables two refinement steps per solve by default, which made every local solve 3–3.7× slower; inside a preconditioner and an eigensolver the backward error of the factorization is sufficient. UMFPACK runs with METIS ordering and no refinement as well (on a 36^3 convection–diffusion matrix: 31% fewer nonzeros in the factors and a 2.1× faster numerical factorization).
- **Block operations in the eigensolver.** The kernel detection solves all probe vectors in one multi right-hand-side call per inverse-iteration step.
- **Small coarse problems on one process** (`-coarse-root-below`): below $n_0 = 10^4$ the

Table 3: Test programs and number of checks with 4 processes in the default build (UMFPACK and MUMPS enabled).

Test	What is compared with a sequential reference	Checks
`test_subdomain`	overlap, halo, A_{ii}, lumping sums, partition of unity, SpMV; symmetry detection	32
`test_preconditioner`	sparse vs dense eigensolver (kernels, eigenvalues, residuals of (6)); RAS/ASM $\times$ deflated/additive against a reference assembled on one process; GMRES	192
`test_reuse`	refactorization and refresh vs fresh setup on time-dependent matrices; reuse policy; time loop	108
`test_factorization`	Cholesky/LDL$^\top$/LU chains, single precision, every coarse solver of the build	97
`test_inexact`	AMG V-cycle as local solver: linearity, determinism, two-level GMRES with the same coarse space	22
`test_stopping`, `test_bicgstab`	OpenFOAM stopping test, GMRES and BiCGStab, iteration budget	26
`test_warmstart`, `test_bubbles`	p_rgh right-hand side, absolute tolerances, warm start; bubble field shared with OpenFOAM	21

gather to rank 0 is cheaper than a distributed solve on G100.

- **Warm-up of the direct solver.** In the first run of two G100 jobs the factorizations of the eigensolver took 14–19 s over the whole run on every process, while a complete rebuild took 1.5 s; the initial setup then entered the reuse policy as the cost of a rebuild and delayed the first rebuild by several steps. The cause was not pinned down. Every run now factorizes a small Laplacian with the local backend before the time loop, untimed, which keeps any such start-up cost out of the measurements and out of the decisions of the policy (initial setup / median rebuild = 0.98–1.03 in all later runs).
- **Fewer reductions in the Krylov methods**: CGS2 in GMRES and fused reductions in BiCGStab (Section 3.4).

3.10 Build, tests and reproducibility

The code is built with a Makefile. The only mandatory external dependencies are an MPI C++17 compiler and METIS; Eigen 3.4.0, Spectra 1.0.1 and AMGCL are header-only and bundled. UMFPACK and MUMPS are enabled automatically when their headers are found, and MKL is enabled with a switch; without them the code is complete, with the Eigen solvers for the local matrices and the coarse matrix on rank 0.

Table 3 lists the nine test programs. They build sequential references from the global matrix, broadcast to every process, and compare the distributed objects with them; every check is combined over the processes, so a failure on one rank fails the test. `make check` runs every test with 1, 2 and 4 processes. A build with AddressSanitizer and UndefinedBehaviorSanitizer ran the whole suite without any memory or undefined-behaviour report.

Every campaign on G100 is one SLURM job script that builds its own copy of the binaries, runs the unit tests on the compute nodes, and records the hash of the sources and the loaded modules next to the results; a Python script turns the raw logs into the report and the plots of the campaign. All the numbers of Section 4 can be regenerated from the files committed in the repository.

Table 4: Table 1 of [1] (GMRES(30), rtol $= 10^{-8}$, `nev`= 300) against this implementation at $\tau = 0.9$, 240 processes. n.c.: not converged in 200 iterations.

	Paper		This work		One-level RAS
Matrix	iterations	n_0/N	iterations	n_0/N	iterations
finan512	6	10	6	22	6
nxp1	20	77	29	95	n.c.
CoupCons3D	20	113	21	98	23
tmt_unsym	6	129	5	128	n.c.
ecology2	6	133	5	130	n.c.
thermal2	26	157	5	139	n.c.
atmosmodj	7	298	14	286	79
G3_circuit	8	279	7	279	n.c.
Transport	9	300	8	300	n.c.
memchip	36	226	8	224	44

4 Results

4.1 Platform and methodology

All runs were made on the thin nodes of CINECA Galileo100: two 24-core Intel Xeon processors (Cascade Lake) per node, 384 GB of memory, InfiniBand interconnect; one MPI process per core and one subdomain per process. On G100 the code was compiled with the OpenFOAM-10 toolchain (Intel MPI 2021), METIS 5.1.0 and MKL 2023.2.0, PARDISO for the local matrices and the MKL cluster solver for large coarse problems. The cost of a solve includes the setup and the updates amortized over the sequence (initial setup + updates + Krylov iterations, divided by the number of solves), measured between two barriers. Where the network produced timing bursts, costs were also compared with median times; the conclusions were the same.

4.2 Reproduction of the reference paper

The experiments of Section 4 of [1] were repeated on 240 processes (instead of 256), with METIS instead of ParMETIS, PARDISO instead of MUMPS for the local factorizations and Spectra instead of SLEPc; the convection–diffusion problem (4.1) of the paper was discretized with upwind finite volumes instead of SUPG finite elements. Table 4 compares the SuiteSparse matrices of Table 1. The paper states $\tau = 0.6$, but its row for G3_circuit coincides with the point $\tau = 0.9$ of its own Figure 2; the comparison is therefore made at $\tau = 0.9$, where the coarse space per subdomain is within 15% of the paper on 8 of 10 matrices.

The iteration counts agree within one iteration on six matrices; nxp1 and atmosmodj need more iterations with a coarse space of similar size, thermal2 and memchip fewer. For non-symmetric matrices the count also depends on the partition and on the random right-hand side, which differ between the two codes. Wherever one-level RAS does not converge, the two-level method does. Figure 5 reproduces the effect of τ on G3_circuit: n_0 follows the paper within 15% except at $\tau = 0.6$ (-26%), and the iterations are 9–29% lower. In strong scaling (Figure 4 of the paper, three-dimensional problem (4.1) with 8×10^6 unknowns) the iterations stay bounded, $16 \to 12$ from 30 to 240 processes, and the time decreases from 187 s to 16.8 s: the superlinear speedup comes from the factorizations and the local eigensolves, whose cost grows faster than linearly with the subdomain size. For the convection–diffusion problem of Table 2 the trends of the paper are reproduced; the coarse space is smaller than in the paper (0.5–0.6× in 2D, 0.2× in 3D), consistently with the smaller stencils of the finite volume discretization, which reduce the lumping, and in 3D with $\nu \leq 10^{-3}$ this costs iterations.

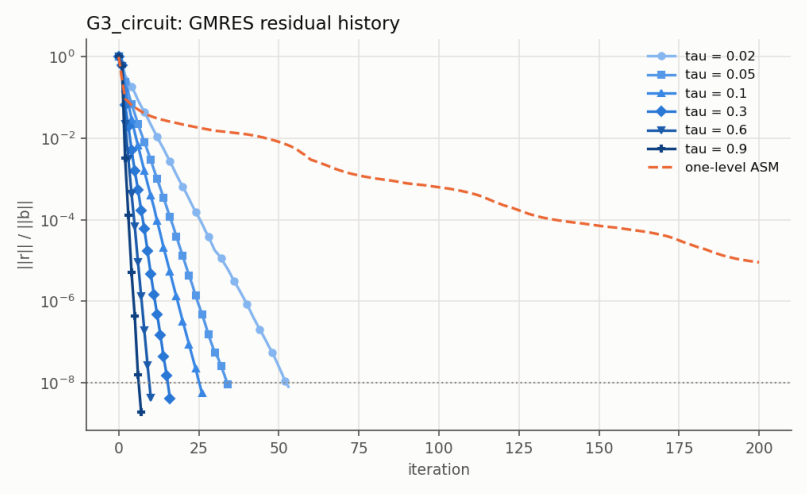


Figure 5: G3_circuit, 240 processes: residual history of GMRES(30) with the two-level preconditioner for the values of τ of Figure 2 of [1], and one-level ASM.

Table 5: Optimization phases on G100: cost per solve, setup and updates included. Last column: before $\rightarrow$ after the phase on the 192^3 mesh with 240 processes, each phase measured in its own job. Details in the `RESULTS.md` files of `test/reuse`, `test/cholesky`, `test/p5` and `test/p7`.

Phase	Change	Outcome (all measured cases)	192^3 [s/solve]
P1	setup reuse: refactorizations, rebuild at break-even (Section 3.7)	5.7–7.4× cheaper than a new preconditioner at every solve	16.0 → 2.62
P2	Cholesky instead of LU for A_{ii}, $\widetilde{A}_{ii}+\sigma I$, A_{00}	−10% to −18%; identical iterations	2.40 → 2.15
P5	small coarse space: cap `nev`= 8 instead of τ alone	−52% to −80% (rebuilds 4–6× cheaper)	2.09 → 0.99
P7	one AMG V-cycle (smoothed aggregation, Gauss–Seidel) instead of the exact factors of A_{ii}	−26% to −56%	1.00 → 0.51

4.3 Optimizations for sequences of systems

The following phases adapted the method to a time loop on the bubbly-flow problem of Section 2.1 (150 bubbles of radius 0.04, density ratio 10^3, three correctors per step, every solve from zero with a new random right-hand side, GMRES(30) to rtol $= 10^{-8}$). Table 5 summarizes them; on the 192^3 mesh (7.1 million unknowns, 240 processes) they reduced the cost of a solve from 16.0 s with a new preconditioner at every solve to about 0.5 s.

Figure 6 shows the effect of setup reuse. Without a coarse space, RAS needs about 800 iterations per solve. With the reuse policy the two-level preconditioner is refactorized at every step and rebuilt when the aging basis has wasted as much time as a rebuild costs: the iterations follow a saw-tooth from 12 to 31–39, and the total cost is the same as that of never rebuilding (`refactor`), whose iterations grow from 12 to 47 in 12 steps and then settle around 30. At the lower density ratio 10 the basis barely ages and the policy rebuilds only twice in 60 steps. Phase P2 changed only the cost of the factorizations: the iteration sequences are identical. Phase P5 showed that a coarse space of 8 vectors per subdomain is enough at this density ratio: every coarse operation becomes cheaper and Arnoldi computes fewer eigenpairs, so a rebuild is 4–6× cheaper and the policy rebuilds more often, which keeps the average iterations unchanged. In P7, one V-cycle of smoothed-aggregation AMG behaves as an exact local solve on these matrices: the first solve after a rebuild needs the same iterations at 192^3 (24.7 against 24.5) and 3.7 more at 256^3, while the

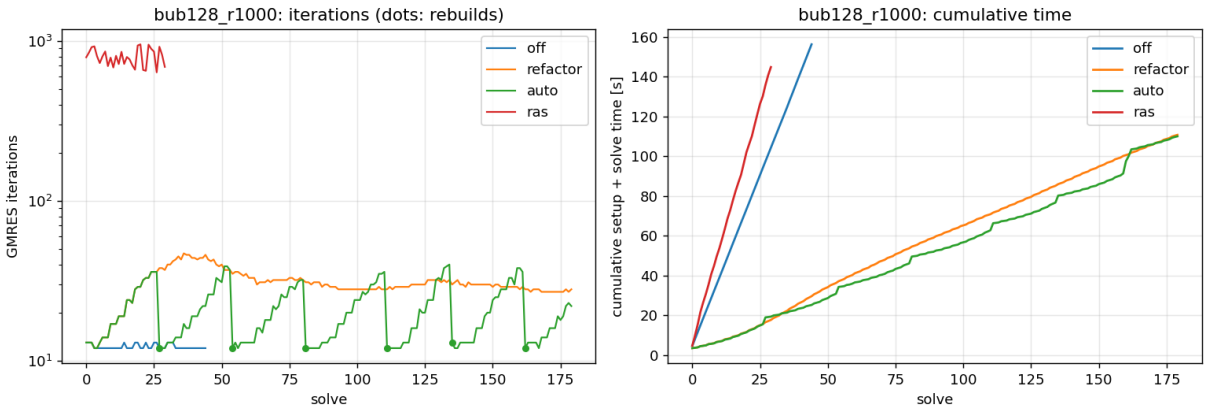


Figure 6: Setup reuse on the bubbles problem (128^3, density ratio 10^3, 240 processes, GMRES to 10^{-8}). `off`: new setup before every solve; `refactor`: numerical refactorizations, basis never rebuilt; `auto`: break-even rule (dots: rebuilds); `ras`: one-level RAS.

local solves are 3.4–4.4× cheaper, since the exact 3D factors have 20–35 times the nonzeros of A_{ii} and a V-cycle reads about $1.5\,\mathrm{nnz}(A_{ii})$ a few times. Ideas that did not pay off were measured and removed: incomplete Cholesky local solvers (not spectrally equivalent: 2.4–8.7× the iterations), other AMG smoothers, reusing the AMG hierarchy across refactorizations, single-precision factors as a default, a larger coarse space at the practical tolerance, warm-started eigenvectors and the reuse of the factorization of $\tilde{A}_{ii}$ between rebuilds.

4.4 Comparison with the GAMG solver of OpenFOAM

Set-up. A small OpenFOAM-10 application, `bubblyPressureFoam`, assembles (1) on a uniform mesh with the bubble field of the generator (a header-only class shared by the two codes) and solves it with the OpenFOAM solvers. Before every campaign the matrix and the right-hand side written by OpenFOAM are compared with those of the generator: they agree to 3.6×10^{-15} relative to the largest entry, and the jobs stop otherwise. This check found a real discrepancy: compiled with the Intel compiler and aggressive optimization, the bubble field of the OpenFOAM application placed 2 of 150 bubbles elsewhere; the field is now compiled with the same compiler as `schwarz2lvl` and linked through a C interface. GAMG runs with the OpenFOAM defaults (`faceAreaPair` agglomeration); the DIC smoother was the fastest OpenFOAM variant in a first comparison against Gauss–Seidel, `algebraicPair` agglomeration and GAMG-preconditioned CG. The two-level preconditioner uses the settings of phases P5 and P7 (`-tau 0.3 -nev 8`, AMG local solves, reuse `auto`). Meshes of 192^3, 256^3 and 320^3 cells (7.1, 16.8 and 32.8 million unknowns) were run on 240, 336 and 432 processes.

Every solve from zero. To $\|r\|_2/\|b\|_2 \le 10^{-8}$ on 240 processes the two-level method needs 51 iterations on average at both 192^3 and 256^3, while GAMG needs 139 and 271 V-cycles: it is 2.0× and 4.3× faster per solve. At the tolerance used by OpenFOAM in a time step (`relTol` 10^{-2} of the normalised residual) the gain is smaller but holds on all nine configurations (Table 6): 1.8–2.7×.

One iteration of the two-level method costs about as much as one V-cycle of GAMG at 240 processes (34.8 against 34.5 ms at 256^3, 68.9 against 68.8 ms at 320^3): the whole gain is in the number of iterations, 2.0–2.9× fewer. From 192^3 to 320^3 GAMG goes from 32 to 69 V-cycles per solve, while the Schwarz iterations right after a rebuild go from 11 to 15 (Figure 8). The two-level method also scales better (Figure 7): the parallel efficiency from 240 to 432 processes is 0.85/0.96/1.18 against 0.56/0.75/0.96 for GAMG (192^3/256^3/320^3), and at 192^3 GAMG does not speed up at all because its V-cycle becomes slower on more processes. In the Schwarz cost,

Table 6: Cost per solve [s] at `relTol` 10^{-2}, every solve from zero, 16 steps × 3 correctors (S: two-level Schwarz with GMRES(30); G: GAMG with the DIC smoother) and ratio G/S.

	240 processes	336 processes	432 processes
192^3	S 0.316 · G 0.567 · **1.79**×	S 0.258 · G 0.570 · **2.21**×	S 0.206 · G 0.563 · **2.73**×
256^3	S 0.986 · G 1.99 · **2.02**×	S 0.724 · G 1.65 · **2.28**×	S 0.573 · G 1.48 · **2.59**×
320^3	S 2.38 · G 4.74 · **1.99**×	S 1.80 · G 3.39 · **1.88**×	S 1.12 · G 2.74 · **2.44**×

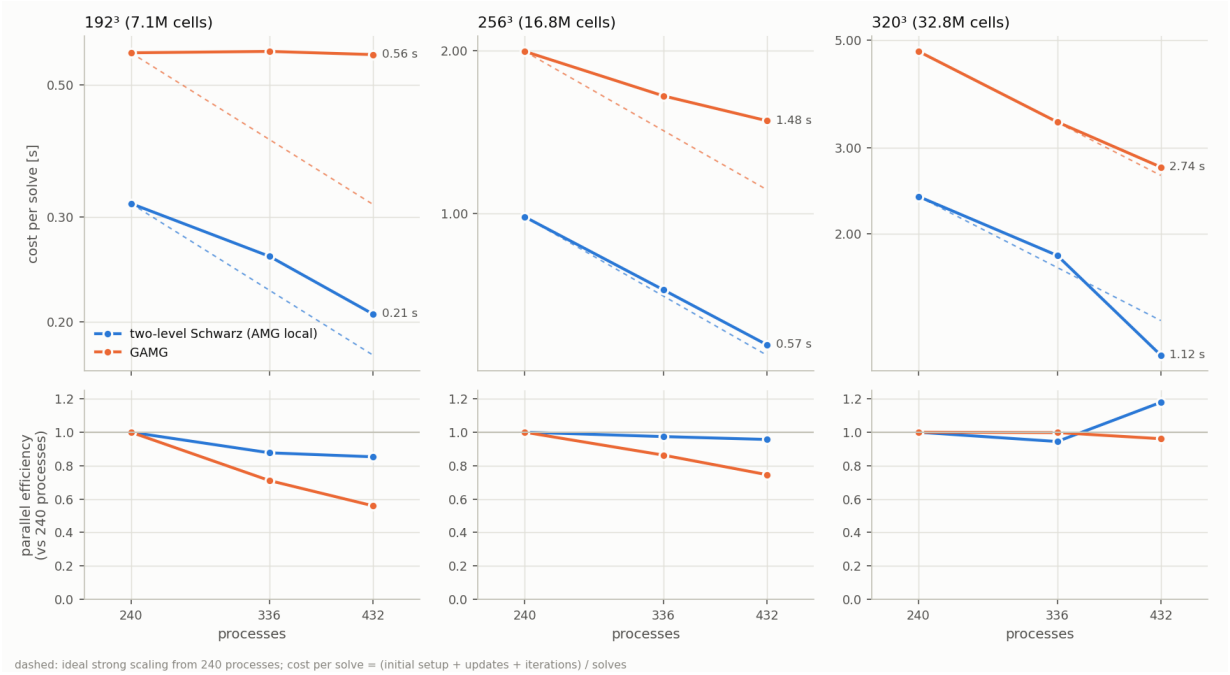


Figure 7: Strong scaling at `relTol` 10^{-2}: cost per solve (top; dashed: ideal from 240 processes) and parallel efficiency relative to 240 processes (bottom).

setup and updates take about 30%, the local AMG V-cycles 21–33%, the coarse correction 11–29% (growing with the number of processes) and the Krylov method itself 17–27%, most of it in the orthogonalization and the stopping test of GMRES.

Krylov method. BiCGStab has a fixed amount of vector work per application of the preconditioner and explicit residuals, which makes the OpenFOAM stopping test nearly free. With the same preconditioner it needs 1.04–1.20× the applications of GMRES, each 0.72–0.85× as expensive, for a cost per solve of 1.02, 0.89 and 0.82× that of GMRES at 192^3, 256^3 and 320^3 (Figure 9). Since it is the algorithm of `PBiCGStab`, a port could use the existing OpenFOAM solver with the two-level Schwarz as its preconditioner. Two further campaigns tuned the coarse space. A looser tolerance of the Arnoldi method (`-eig-tol 1e-2 -eig-ncv 12`) needs 27–29 applications of the operator instead of 42 and reduces the initial setup by 18%, with the same iterations after every rebuild: the bubble modes have large, well separated eigenvalues and converge in a few steps, while resolving the cluster just above $1/\tau$ to high accuracy adds cost and nothing to the preconditioner. A larger coarse space, 12 or 16 vectors per subdomain instead of 8, reduced the iterations by 8–22% but increased the cost per solve by 4–20%, because the coarse correction and the setup grow with `nev`; the gain in iterations also vanishes on the finest mesh, and the coarse space ages just as fast. With BiCGStab, `nev`= 8 and the looser Arnoldi, the setting used in the next campaign, the two-level method is 2.0, 2.6 and 2.5× faster than GAMG on 240 processes at 192^3, 256^3 and 320^3.

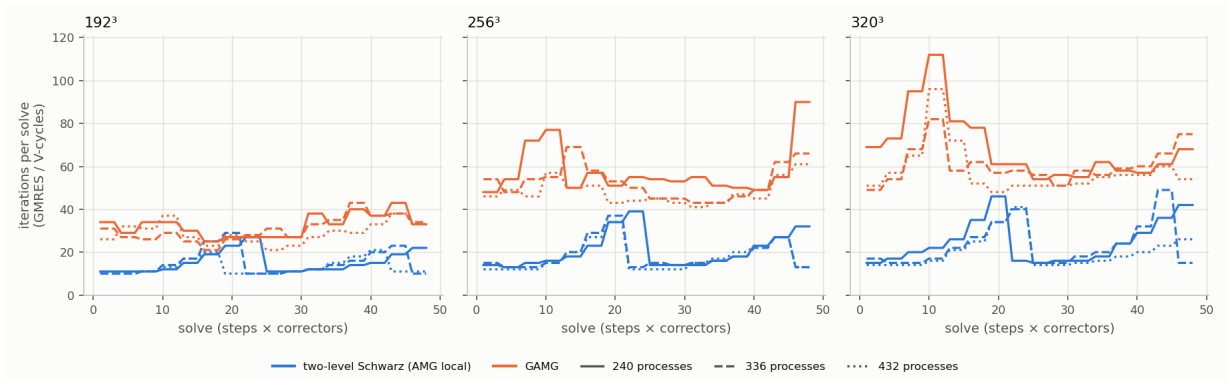


Figure 8: Iterations of the two-level Schwarz preconditioner (GMRES) and V-cycles of GAMG for every solve of the time loop. The saw-tooth of the Schwarz iterations is the aging of the coarse space between two rebuilds.

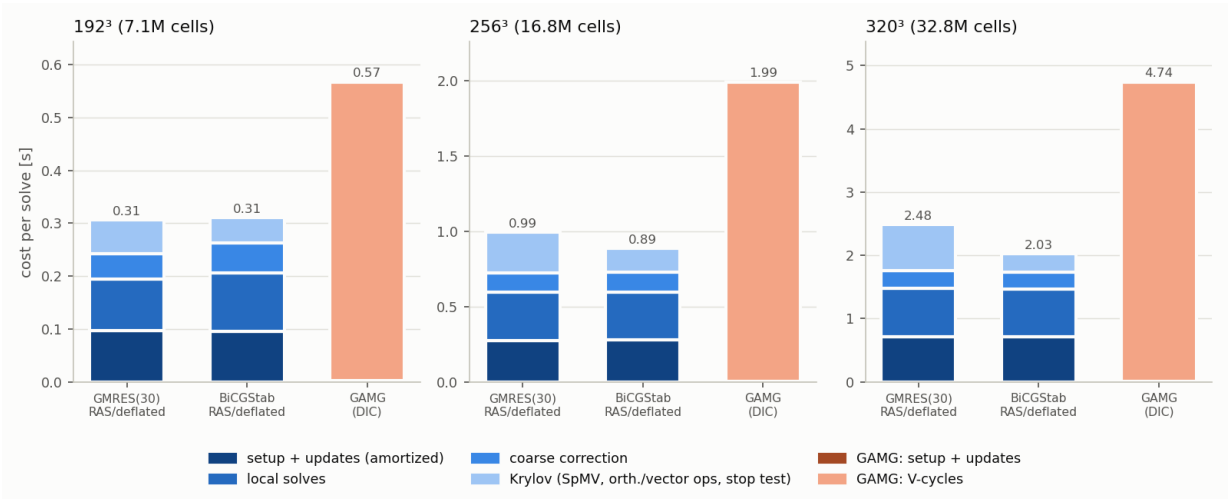


Figure 9: Cost per solve at 240 processes, `relTol` 10^{-2}: two-level Schwarz with GMRES(30) and BiCGStab, split into setup and updates, local solves, coarse correction and Krylov work, and GAMG.

A PIMPLE-like time loop. The comparisons above solve every system from zero with a right-hand side constant on every bubble, which projects almost entirely on the near-kernel modes of Figure 1. An OpenFOAM time loop differs in three ways: the source of the p_{rgh} equation of a volume-of-fluid solver lives on the interfaces and sums to zero over every bubble, every solve starts from the previous solution, and only the last corrector of a step is converged tightly. The last campaign reproduced these conditions: right-hand side $-\nabla \cdot \phi_g$ of interFoam for the fluid at rest, warm starts, the first two correctors to `relTol` 0.05 (or tolerance 10^{-7}), the last one to tolerance 10^{-7}, the settings of the `p_rgh`/`p_rghFinal` controls of the OpenFOAM tutorials. The two codes solve the same sequence: the initial normalised residual of the first corrector of every step agrees between them to 3×10^{-5}. A decision rule was written before the run: a port is justified if the two-level method is at least about 1.5× faster than the best OpenFOAM variant at both process counts.

Table 7 and Figure 10 summarize the result. The two-level method is faster than GAMG in five of six configurations (1.17–1.81×) and equal at 192^3 on 240 processes. The gain now lives entirely in the last corrector, where it is 2.0–4.1×; on the first two correctors, 1.3 decades from a good initial guess, GAMG is 1.2–1.7× cheaper, because little of the error lies in the bubble modes and the coarse correction is paid for at every application without being needed. Setup and rebuilds are 39–43% of the Schwarz cost: the iterations became cheaper, the setup did not. The one-level RAS keeps up on the first correctors but needs 420–700 applications on the last one,

Table 7: PIMPLE-like loop (24 steps × 3 correctors, warm start): cost per time step [s], all correctors, setup and updates included, and average iterations of the last corrector (applications of the preconditioner / V-cycles). Schwarz with BiCGStab, `nev`= 8, AMG local solves; GAMG_DIC is the fastest OpenFOAM variant.

		Cost per time step [s]				Last corrector
Mesh	Proc.	Schwarz 2-level	RAS 1-level	GAMG_DIC	GAMG/Schwarz	Schwarz / GAMG
192^3	240	0.605	3.27	0.581	**0.96**×	22.5 / 35.2
256^3	240	1.68	11.2	1.97	**1.17**×	29.8 / 56.2
320^3	240	4.30	26.2	5.08	**1.18**×	38.4 / 71.9
192^3	432	0.358	1.90	0.647	**1.81**×	19.3 / 33.8
256^3	432	0.911	6.85	1.51	**1.66**×	24.0 / 53.0
320^3	432	2.06	15.2	2.79	**1.35**×	30.5 / 59.7

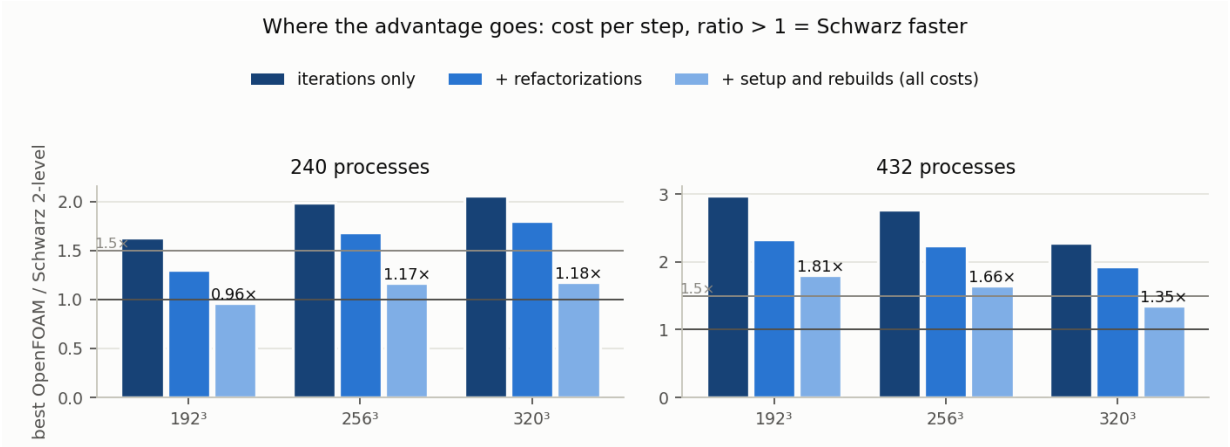


Figure 10: PIMPLE-like loop: ratio of the cost per time step of the best OpenFOAM variant to that of the two-level Schwarz preconditioner, built up phase by phase (iterations only; with refactorizations; with setup and rebuilds). Horizontal lines: parity and the 1.5× threshold of the decision rule.

2.9–5.7× the cost of GAMG: with p prescribed only on the top face, the low-energy modes span the whole domain, and only the coarse space makes the tight final tolerance affordable. By the rule written before the run, a port is not justified as it stands: the threshold is reached only on 432 processes for the two smaller meshes.

4.5 Interpretation

The measurements are consistent with the mechanism of Section 2.1. GAMG pairs cells by face area; on a uniform mesh its aggregates ignore the interfaces and straddle them, so the modes that are nearly constant on each bubble are left to the smoother, which propagates information by a few cells per sweep, while a bubble spans 15 cells at 192^3 and 26 at 320^3: the number of V-cycles grows with the mesh. The local eigenproblems of the two-level method select, in every subdomain, precisely the low-energy modes, whatever the shape of the bubble pieces it contains, so the iterations grow only mildly. The price is that the coarse space follows the bubbles: within 8–12 time steps (4–6 cells of rise) the iterations of the last corrector grow by a factor 2.9–4.0 and the basis must be rebuilt, at a cost that GAMG, whose agglomeration ignores the coefficients, never pays. The PIMPLE-like campaign also shows that a right-hand side constant on the bubbles, solved from zero, overstates the difficulty of the problem for GAMG: in a small local test 99% of the energy of the initial error lay in the ten lowest modes with that right-hand side, and less than 0.1% with the p_{rgh} source and a warm start.

5 Discussion and future work

Limitations of the study. The comparison with GAMG covers one problem (a bubbly flow with density ratio 10^3) on uniform meshes, with OpenFOAM 10 while the target would be a more recent version, and different decompositions in the two codes (scotch for OpenFOAM, METIS here). Each configuration of the last campaign was run once; earlier campaigns showed a run-to-run spread of ±3–4%. The source term of a real flow also contains the divergence of the predicted velocity, which is not confined to the interfaces. Finally, the matrices of `schwarz2lvl` are generated on rank 0 and this time is not counted: it is a property of the test harness (OpenFOAM assembles in parallel), but it makes a Schwarz job longer than a GAMG job in wall time.

Limitations of the implementation. Real matrices only; one subdomain per MPI process; the global matrix is read on rank 0 during the decomposition; local indices and the interfaces to the coarse solvers (MUMPS, MKL with 32-bit integers) are limited to 2^{31} entries. For non-symmetric or indefinite matrices outside the class of [1] the coarse space carries no guarantee.

Next steps. The part of the cost left to reduce is keeping the coarse space up to date: rebuilds and the aging between them account for most of the difference between the cold-start and the PIMPLE-like results. The modes that age are those nearly constant on bubbles that have moved; a natural candidate is to keep the eigenvectors of the last rebuild and add, at every step, indicator vectors of the gas regions of each subdomain, computed from the current coefficients in $O(n_i)$ and assembled into A_{00} at every refactorization. A distributed generation of the matrix values would more than halve the core-hours of the Schwarz test runs. If the preconditioner is ported to OpenFOAM, the natural form is an `lduMatrix::preconditioner` used by the existing `PBiCGStab`, with the subdomain structure, the coarse space and the reuse policy stored in a persistent object (e.g. a `MeshObject`, as GAMG stores its agglomeration), and `LocalSystem` filled directly from the processor matrix.

6 Conclusions

The algebraic two-level Schwarz preconditioner of Al Daas, Jolivet and Rees was implemented in C++17/MPI as a library whose only input is the set of rows owned by each process, the data an OpenFOAM processor has. The implementation reproduces the experiments of the reference paper on 240 processes, and its distributed components are verified against sequential references by about 500 automated checks.

On the pressure equation of a bubbly flow with density ratio 10^3, the setup was adapted to sequences of matrices with numerical refactorizations and a break-even rebuild rule, a small coarse space and AMG local solves, which together reduced the cost of a solve about thirty times with respect to rebuilding the preconditioner at every solve (16.0 → 0.51 s on 7.1 million unknowns and 240 processes). On identical matrices of up to 32.8 million unknowns the two-level method is 1.8–2.7× faster than the GAMG solver of OpenFOAM when every system is solved from zero to the OpenFOAM tolerance, 2.0–4.3× to 10^{-8}, and it scales better. The gain is algorithmic: one iteration costs as much as a V-cycle, but the spectral coarse space captures the near-kernel modes of the bubbles that the geometric agglomeration of GAMG cannot represent. In a PIMPLE-like loop with warm starts the advantage shrinks to 0.96–1.81× per time step and is concentrated in the tightly converged last corrector, while setup and rebuilds become 40% of the cost. By the criterion fixed in advance, a port to OpenFOAM is not yet justified; a coarse space that follows the bubbles at the cost of a refactorization is the step that could change this conclusion.

References

[1] Hussam Al Daas, Pierre Jolivet, and Tyrone Rees. Efficient algebraic two-level Schwarz preconditioner for sparse matrices. *SIAM Journal on Scientific Computing*, 45(3):A1199–A1213, 2023. doi: 10.1137/22M1469833.

[2] Scott P. MacLachlan, J. M. Tang, and Cornelis Vuik. Fast and robust solvers for pressure-correction in bubbly flow problems. *Journal of Computational Physics*, 227:9742–9761, 2008.

[3] Henry G. Weller, Gavin Tabor, Hrvoje Jasak, and Christer Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12(6):620–631, 1998.

[4] Victorita Dolean, Pierre Jolivet, and Frédéric Nataf. *An Introduction to Domain Decomposition Methods: Algorithms, Theory, and Parallel Implementation*. SIAM, Philadelphia, 2015.

[5] Youcef Saad and Martin H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.

[6] Henk A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 13(2):631–644, 1992.

[7] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.

[8] Gaël Guennebaud, Benoît Jacob, et al. Eigen v3. `https://eigen.tuxfamily.org`, 2021. Version 3.4.0.

[9] Timothy A. Davis. Algorithm 832: UMFPACK V4.3—an unsymmetric-pattern multifrontal method. *ACM Transactions on Mathematical Software*, 30(2):196–199, 2004.

[10] Intel Corporation. Intel oneAPI Math Kernel Library: PARDISO and parallel direct sparse solver for clusters. `https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html`, 2023. Version 2023.2.0.

[11] Denis Demidov. AMGCL: An efficient, flexible, and extensible algebraic multigrid implementation. *Lobachevskii Journal of Mathematics*, 40(5):535–546, 2019.

[12] Patrick R. Amestoy, Iain S. Duff, Jean-Yves L'Excellent, and Jacko Koster. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[13] Yixuan Qiu. Spectra: C++ library for large scale eigenvalue problems. `https://spectralib.org`, 2022. Version 1.0.1.